Table of contents

Series 7: DPLL with Theories and Array Theory

Methods

DPLL(T)

A decision method for first-order formulas combining the classical DPLL algorithm (for the propositional part) with a theory solver (T) to verify consistency of literals in a specific theory (e.g., integers, EUF, arrays).
Subcategories:

- **Propositional DPLL**: assignment of truth values, unit propagation, backtracking.
- **Theory integration**: the theory solver validates or invalidates partial assignments based on theory constraints.

Linear Integer Theory

Theory allowing reasoning about linear inequalities and equalities between integer variables. Used here to interpret literals such as $a + 1 \leq b$.

EUF Theory (Equality with Uninterpreted Functions)

Equality theory extended to uninterpreted function symbols. Axioms are reflexivity, symmetry, transitivity, and congruence: if $x = y$ then $f(x) = f(y)$.

Array Theory

Theory specific to read (`select`) and write (`store`) operations on arrays. Fundamental axioms:

1. Read after write at the same index.
2. Read after write at a different index.
3. Extensionality: two arrays are equal if all their elements are equal.

Formalization in First-Order Logic

Translation of informal statements (program properties, data structure properties) into quantified first-order logic formulas, usable later by automated solvers.

Code Equivalence Verification

Comparison of two code fragments by expressing their semantics using a theory (e.g., arrays) and searching for a logical condition guaranteeing equality of results.

Reminder

- **DPLL algorithm**: recursive search for a model of a propositional formula, with unit propagation and backtracking.
- **Array axioms**:
  - $\forall a, i, e.\ \text{select}(\text{store}(a, i, e), i) = e$
  - $\forall a, i, j, e.\ i \neq j \Rightarrow \text{select}(\text{store}(a, i, e), j) = \text{select}(a, j)$
  - $\forall a, b.\ (\forall i.\ \text{select}(a, i) = \text{select}(b, i)) \Rightarrow a = b$
- **Congruence in EUF**: $\forall x, y.\ x = y \Rightarrow f(x) = f(y)$
- **Equivalence condition for two successive stores**:
  $\text{store}(\text{store}(a, i, b), j, c) = \text{store}(\text{store}(a, j, c), i, b)$ iff $(i \neq j) \vee (i = j \wedge b = c)$

Illustrative Examples

Example 1: DPLL with Integer Theory

Context:

We have a formula $\phi = (p \vee q) \wedge (\neg p \vee r) \wedge (s \vee t) \wedge (\neg r \vee s \vee u)$, where each propositional variable is associated with an integer constraint (e.g., p corresponds to $a + 1 \leq b$). We search for an assignment satisfying ϕ and the theory constraints.

Application:

1. **Propositional DPLL**: start with assignment $p \leftarrow \text{True}$.

   - $\phi[p] = r \wedge (s \vee t) \wedge (\neg r \vee s \vee u)$.
   - Choose $r \leftarrow \text{True}$: $\phi[p, r] = (s \vee t) \wedge (s \vee u)$.
   - Choose $s \leftarrow \text{True}$: $\phi[p, r, s] = \text{True}$.
   - Propositional model: $I = \{p \leftarrow \text{True}, r \leftarrow \text{True}, s \leftarrow \text{True}\}$.

2. **Integer theory verification**:

   Corresponding constraints are:

   - $p : a + 1 \leq b$
   - $r : a > b + c$
   - $s : b > 0$

   We must find integers a, b, c satisfying these three inequalities.

   Choice: $a = 10,\ b = 11,\ c = -2$

   - $10 + 1 \leq 11$ true,
   - $10 > 11 + (-2)$ i.e., $10 > 9$ true,
   - $11 > 0$ true.

   Therefore the model is valid in integer theory.

Example 2: Array Theory and Code Equivalence

Context:
We compare two write sequences in an array:

- `array1 = store(store(a,i,b), j, c)`
- `array2 = store(store(a,j,c), i, b)`

We want to know when `array1` equals `array2`.

Application: 1. **Case** $i \neq j$:

Writes are at distinct indices, order does not matter. Both sequences produce an array where:

- at index i: value b,
- at index j: value c,
- elsewhere identical to a.

Therefore equality.

2. **Case** $i = j$:
   Both writes are at the same index. The last write overwrites the first.

   - `array1` writes b then c: final result c at index i.
   - `array2` writes c then b: final result b at index i.

   For equality, we need $b = c$.

3. **Logical condition**:
   The equivalence condition is therefore:

$$(i \neq j) \vee (i = j \wedge b = c)$$

   This can be written in FOL with array theory and equality.

4. **Why3 verification**:
   This condition can be encoded as a precondition in Why3 to verify that the two arrays are equal. If the condition is satisfied, the verifier will display "green V".

Synthesis

This series illustrates the combined use of logical satisfaction methods (DPLL) with first-order theories (integers, EUF, arrays) to model and verify program properties. The key is separating the propositional part from the theory part, and using specialized solvers for each theory. Formalization in first-order logic and the use of tools like Why3 allow automating verification of equivalence or validity conditions.